

Operating Systems and Networks

OS is the middleware b/w hardware and the application / user programs. It provides abstraction to the users for various system things like the memory, context switching, storage, and other H I D

Virtualization :- Processes have the illusion of it being the only process with access to its own unique memory.

Concurrency :- Multiple processes can run simultaneously

Persistence :- OS also handles interactions with the disk and performs storage management.

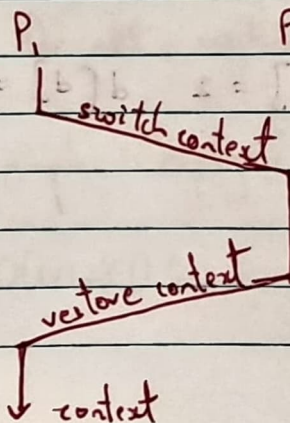
Process Virtualization:-

Process :- Running programs $\rightarrow$ set of instructions

Program $\xrightarrow[\text{build}]{\text{compile and}}$ Executable $\xrightarrow{\text{execute}}$ Running Process

Only running process programs consume resources.

Each process has its own CPU. Multiple processes can run on the same CPU with the help of context switching (virtualization of the CPU)



- (i) Load program into memory
- Init prog on disk
 - OS does lazy loading $\rightarrow$ Load only relevant stuff

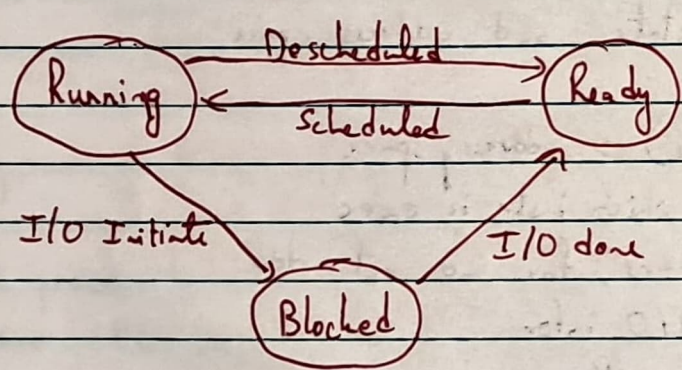
- (ii) Allocate runtime stack:-
- Use for local var
 - Func params and return values

- (iii) Creation of prog heap:-
- Used for dynamic data like malloc and free

- (iv) Basic file setup
- Stdin, Stdout, Stderr

State of a process:- Multiple states like "running", "suspended", "blocked", "ready"

$\downarrow$ $\downarrow$
 waiting due to context switching
 for I/O



The processes are stored in a list called process list
Process Control Blocks are stored in the list. (structs)

- $\hookrightarrow$ Proc id
- $\hookrightarrow$ state
- $\hookrightarrow$ Address Space

Kernel:- Part of the OS that handles direct access to the hardware.
Monolithic, Hybrid and Micro.

The OS should be able to create a process, kill a process, ^{make process wait} wait, suspend or find the status of a process.

APIs allow different programs to communicate with each other. Provides a software interface.

System calls are methods by which the OS provides some functions that can be leveraged by user programs.

POSIX (Portable Operating Systems Interface)

→ Standard set of system calls that an OS must implement

⊗

This ensures uniformity for languages to not worry about system calls syntax.

There are two modes of execution:- User mode and Kernel mode
normal access for the user

↓
Higher privilege

The first process that is run on boot is the init process. It is the ancestor of all processes.

Fork:- A new independent child process is created. Execution of two happens simultaneously.

Wait:- calls blocks/bubbles in parent until child terminates.

If proc terminates in the absence of ~~the~~ wait → zombie process. Init adopts orphans and reaps them.

KV has doll-dole merger, but in direction

Date: . . .

Exec:- Load a different executable to the memory.
Fork & exec are used in conjunction to retain control.

We need to ensure that processes do not do any thing some thing unexpected, and also stop and switch b/w process.

Hardware Support:- Have some low level mechanisms.

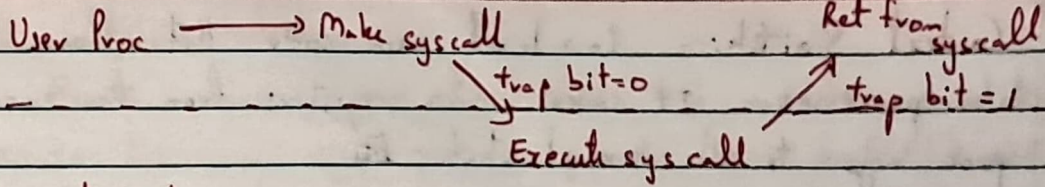
Limited Direction Execution:- Every process gets some time to run on the CPU, after which the OS shifts the context to another process.

Interrupt Descriptor Table:- Mapping b/w id and actual location of the system calls. Useful in switching from User to Kernel mode, as it obfuscates the true location of syscalls.

Trap instr:- Special kind of instr to switch from user to kernel mode. It raises the privilege from User to Kernel mode. Return-from-trap allows switching back into user mode and returns to the calling program. The normal routine is interrupted. Context must be saved in the kernel stack. The OS has multiple kernel stacks, one per process.

- CPU goes to higher privilege [Kernel Stack]
- Switch to kernel stack and save context
- Look up in IDT and jump to Trap handler
- Perform privileged instr.
- Call return-from-trap
- Return to User Mode.

We use Kernel Stack for safety reasons.

User ModeKernel Mode

Interrupt:- Signal that conveys that "something" has happened due to unexpected events. I/O interrupt, Console Interrupt, etc. Can be Hardware or Software.

Trap:- Software interrupt caused by exceptions or syscalls.

The trap instr is ~~also~~ called by the syscall.

Switching b/w Processes(i) Cooperative [Non-preemption Approach] :-

- Wait for system call.
- Process transfers the control to the CPU by making a sys call
- There can be misbehaving processes → divide by zero, etc. We then Trap to OS → Process gets terminated
- Inf loops and termination are issues → Reboot! for getting control.

(ii) Non-cooperative [Preemptive Approach] :-

- OS gains control by using interrupts.
- Timer interrupt → Every x time units, halt process, invoke interrupt handler and OS can choose to change the process. OS starts timer on boot
- The CPU with the help of Scheduler chooses to continue current process post interrupt
- (or)
- Perform context switch

Context Switch:- Low level asm code that allows switching b/w processes. It saves a few registers from the executing proc reg to the kernel stack.

What if during handling of one interrupt, another interrupt occurs?

Disable other interrupts!

Create sophisticated locking mechanisms to safeguard that block of memory from concurrent access.

Scheduling Processes Policies:-

To evaluate the quality of a scheduler, we need estimates like the no. of proc in the queue, what is their goal/purpose, how much time do they require etc, I/O reqs, Mem reqs etc.

Each process that is in the queue for execution $\rightarrow$ Job

Assumptions:-

- $\rightarrow$ All processes run for the same time
- $\rightarrow$ Arrive at same time
- $\rightarrow$ Use only CPU
- $\rightarrow$ Run time is known.

Performance Metric:- Turnaround time

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

Fairness:- Guarantee that all processes at least gets the chance to execute.

Fairness & Performance may not go hand-in-hand

All assumptions intact: -

Assume 3 proc arrive at the same time and have same estimated exec run time. We just pick a process (rand or FCFS) and serialise.
 But if we relax the assumption that all proc take same time, FCFS is not a great idea. (~~convoy~~ convoy effect)

We then choose to short schedule the shortest job first. [SJF]

But this is problematic if the processes can arrive at various times.
 So what we do is when a new job arrives, we schedule the job with the Shortest Time to Completion First
 ↳ Preemptive

Perf Metric:- Response Time

$$T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$$

LLMs stream the output instead of sending it in one chunk just so that the user feels that the response time is minimal

A better idea is to run all process for a short time and then preempt. Response time is optimised, but turnaround time is bad.



Round Robin:- Run jobs for a time slice → switch to next job → repeat till all are done.

Time slice is the short time for which a process is run. Uses timer interrupts. There is an Overhead of Context Switch.

Tradeoff b/w Turnaround time and Response Time

Turnaround only cares about completion while Response Time cares only about performance / interactivity / ping.

Round Robin (naive) doesn't work well with I/O

During I/O, the process goes into a blocked state and then wakes up on an interrupt.

We can instead leverage STCF along with RR by not executing other processes while one process is performing I/O

What if we do not know the length or expected time of completion of the job? (We usually don't)

So let us prioritize?

Multi Level Feedback Queues

→ Reduce Turnaround times by running shortest job first

Use n -queues for n -different priority levels.

Use the priority to decide which job to run

Job with higher priority $\Rightarrow$ Job on a higher queue.

Scheduler keeps updating priority based on observed behaviours.

One of the metrics used is the time taken for execution within the time-slice allocated

→ Interactive jobs $\Rightarrow$ Higher priority (Keep returning to CPU)

→ CPU intensive $\Rightarrow$ Reduce priority

→ If Priority (A) > Priority (B) $\rightarrow$ Run A

→ Else if $P(A) = P(B) \rightarrow$ RR A & B

R R R

Every new job is placed at high priority.

- (i) If job uses entire time slice, reduce priority.
- (ii) If Elif key at same priority.

Incorporating I/O.

Assume there is a highly interactive job that relinquishes CPU later s.t. its priority does not change.
Long term jobs suffer.

Starvation - Long running jobs never get CPU time if there are many interactive jobs.

Game the Scheduler :- Processes can game the scheduler to get more time than what is fair, by using I/O to relinquish the CPU before time slice is over. This results in the priority of the process not changing.

Programs may also change behaviour over time. There might be CPU intensive phases b/w phases of interactivity.

Priority Boost! :- Every x seconds or so, bring all processes to the highest priority queue or increment their priorities by 1.
This ensures that there is no starvation.
It also takes care of changed behaviours.

To reduce gamification of the scheduler, we perform reverse priority boost by pushing all high priority processes into lower priorities.

Too small $S \rightarrow$ interactive proc doesn't get enough CPU
Too large $S \rightarrow$ starvation.

We have have different time slices for different priority queues

We're doing Networks?!

At the most basic level, networks are hardware (cables)

The process raises an interrupt, asking the OS to transmit the data to another system. The hardware at the other system raises an interrupt and lets the OS₂ access the data.

We need interface b/w process & network, and b/w-network components.

Host:- Any device that can send or receive traffic.

Hosts can be client or server.

The roles are not ME, each can be the other.

Client:- Any host requesting for resources on the network.

Server:- Any host providing resources on the network.

Hop:- A single traversal

↳ Public & Private

We need the IP address to request a server.

↳ Gives the location of a point on the network.

When client sends data, it sends both source and destination IP

Repeater:- Repeaters repeat signals to increase signal strength

Floods the network



Hub/Bridge:- Multi-Port repeaters. Broadcasts the data

↳ Intelligent hubs. Lets all hosts know what hosts exist on the other side, and forwards data to the correct host.

broadcasts → Still floods while we communicate from one side to the other.

Switches:- More intelligent bridges. unicast data rightly. Have switching tables.

Router:- Used to connect b/w networks. They route the data to the correct network. Routers store all routes in a routing table. They provide security, filtering and redirection. They have IP addresses and specific ones pertaining to the connected networks.

Access Point:- Wireless switches

Port no.s are associated with processes

Networking Layers:-

Reduces complexity and provides abstraction.

OSI model:

Application → Provides network services to applications.

Presentation → Ensures the data is in an understandable format

Session → Es Manages connection b/w diff. devices

Transport

Network

Data Link

Physical → Data in the form of 0/1. Smith has to transfer bits from one layer to another, over physical media.

Data Link → Hop to Hop connectivity. Responsible for creating link b/w two ~~correctly~~ directly connected nodes, error correction. MAC addresses are used here.

Network Layer → Manages routing through routes in a large network. Uses IP addresses to perform tasks like logical Addressing, Rtt selection and Packet Switching.

Transport Layer:- Ensures data transfer is reliable, sequential and ~~order~~, manages flow control and error correction. Uses TCP/UDP.

4 Layer Model.

Application - HTTP, SMTP, RTP, DNS

Transport - TCP, UDP.

Internet - IP, ICMP

Network/Link Layer - DSL, Ethernet, 802.11, SONET

Transport Layer:-

Service to Service Delivery:-

- (i) MUXing & DEMUXing
- (ii) The addressing scheme is ports
- (iii) TCP → Transport Control Protocol → Reliable
UDP → User Datagram Protocol → Efficient

- Client sends a request to the server
- The server provides a reply based on the request made by client

MUXing →

Software component in the OS that supports network calls is the Protocol Stack

Drivers are the interface b/w hardware and the OS

Socket is the abstraction provided to a process by the OS to connect to the network

The network service API is used to write all network applications.

Streams:- Reliable (connection oriented)

Datagrams:- Unreliable (connection less)

Allows applications to attach to the network at different ports.

Socket API

`socket()` :- Creates a new socket of a certain type and returns file descriptor

`bind()` :- Also Associates the socket with an IP & port

`listen()` :- For server sockets, allows listening for incoming connections

`accept()` :- For server sockets, it waits for client to connect & ret the new f.d.

`connect()` :- For client sockets to create a connection with the server

`send()/receive()` :- Transmit data

`close()` :- Close the connection

Appln process is identified by the triple (IP, Protocol, Port)
16-bit integers $\swarrow$
that process listens.

Servers oft bind to well known ports

Clients are assigned ephemeral ports that are chosen by the OS temporarily.

Connectionless (UDP):- $\rightarrow$ Unreliable at L₄

(i) UDP socket identified by 2-tuple (Dest IP, Dest Port)

Flow control:- Transmission rate can be controlled by TCP, not by UDP

Overhead:- TCP header = 20 B, UDP header = 8 B

Chunk header into 16 bit segments, ^{add carry back} add them up and perform 1's complement to create a checksum

Receiver performs the same steps and compares against the checksum

• TCP:-

Three way Handshake:-

TCP ~~now~~ ensures right transfer of data
UDP just keeps sending data assuming that someone is receiving
 $\rightarrow$ Guarantees the receiver receiving the data

TCP Header :- Source & Destination port no.

Window $\rightarrow$ Flow control

Checksum

Offset

Urgent Pointer

Reserved

TCP options

C|E|U|A|P|R|F|S bits

Sequence Number

$\hookrightarrow$ Check out for self

Ack Number

Flow control b/w sender & receiver

Congestion control ~~for~~ for the whole network

Seq no:- Used to order / sequence data

Ack no:- Used by receiver to inform the sender how many bytes were received

Round Trip Timeout:- Re transmission timeout. The time before a packet is retransmitted after which the process retransmits data, assuming data loss.

both for ack & sender

Along with the timer, there is a cache which stores the data that just been transmitted. It gets cleared upon acknowledgement.

Time b/w Transmission to Ack

$$\text{Est RTT} = (1 - \alpha) * \text{Est RTT} + \text{Sample RTT}$$

$$\text{Dev RTT} = (1 - \beta) * \text{Dev RTT} + \beta * |\text{Sample RTT} - \text{Est RTT}|$$

$$\text{Timeout interval} = \text{Est RTT} + 4 * \text{Dev RTT}$$

EWMA of deviation of sample RTT from Est RTT, $\beta = 0.75$

Est RTT + safety margin

Est wtd

moving avg $\rightarrow \alpha = 0.25$

Instead of ACKing for every transmission, TCP does delayed ACK, waiting for N seconds or after 2 segments

Deadlock is prevented by having a timer on the receiver's end as well

Dynamic update of window size will enable flow control.

If Window size 0 $\rightarrow$ buffer is full

Seq no $\rightarrow$ pick any randomly and increment during data exchange

Three Way Handshake :- Consists of four events

- Process 1. M_1 sends a connection request with SYN bit set and seq no. X
- Process 2. M_2 ack and sends back ACK with $x+1$
- Process 1. M_2 sends a request with SYN bit set & seq. no Y
- Process 1. M_1 ack and sends back ACK with $Y+1$

Closing Connection :- Two ways → FIN & RST flags

FIN is four way, cause Proc 1 sends FIN, Proc 2 ACKs, Proc 2 finishes proc working on data, Proc 2 sends FIN and finally Proc 1 ACKs and the connection closes.

RST is graceless → Just set RST to 1 & close the connection.

The U bit in the options is used to convey priority of some part of the information, & the Urgent Pointer points to the loc of the imp data.

But what about Memory?

Memory Virtualization :-

The OS, its code & data resides in one part of the memory, while the rest can be used by other processes. Since multiple processes are running, no proc has access to the whole memory, we have to fall back on virtualization.

Keep process in memory?

Each process is given a dedicated location with fixed size

Virtual Address Space:-

Easy to use abstraction:-

Address Space $\rightarrow$ Program Code, Heap, Stack
Dynamic malloc $\leftarrow$ $\rightarrow$ Free call during runtime

Both Stack & Heap resize during runtime.

There is only one physical memory, the process still feels like its memory starts at address 0, thanks to virtualization by the OS.

OS allocates memory and tracks the location of the process.

The translation b/w Virtual Addresses and Physical Addresses is done by the Memory Management Unit.

Goals of Virtualization

Transparency:- Illusion that physical memory is not visible to any process

Efficiency:- Minimize overhead

Protection:- Each process should run in its own safe space.

Assumptions (Once again):-

- Address space is placed contiguously in the physical memory
- Size of the address space is less than the size of the physical memory
- Each address space is of exactly the same size

Base and Bounds Approach:-

Two registers in the CPU ~~in the MMU~~ (in the MMU)

Base Registers

Bounds Registers

↓

OS decides the location in physical memory → written into base register

Address Translation → Process of transforming VA to PA → $PA = VA + \text{Base}$
The assignment of physical address is done dynamically at runtime.

There is only one pair of base and bound registers in the MMU and is stored and swapped during the context switch

If out-of-bounds access occurs → OS raises an exception
SIGKILL ←

↳ Issues with B&B → Fragmentation issues

Segmentation → Generalised B&B:-

Segment:- Contiguous portion of the address space of a particular length.

We have one base and bounds per logical segment of the address space.

Conventionally, address space $\rightarrow$ code, stack & heap

Segmentation allows the segments to be placed anywhere.

Only used memory is allocated in physical memory $\rightarrow$ allows allocating large address space.

How do we map?

(Check slides, I need sleep T_T)

Identifying Segments:-

Explicit:- Use the first two bits to identify segment, and the rest are offset bits.

Implicit:- Use logic (var $\rightarrow$ stack, etc)

$\hookrightarrow$ Generated by PC $\rightarrow$ Code

$\hookrightarrow$ Generated by ESP $\rightarrow$ Stack

$\hookrightarrow$ Else Heap

Bound Checking:-

Ensure abs -ve value of offset is less than segment size.

The registers for storing these values are called segment registers.

Hardware introduces support in the form of protection bits.

Code segments can be R/O ONLY and the hardware can check if address is within bounds and permissible.

$$\begin{array}{r} 2[2] \\ 2[0-1] \\ 2[5-0] \\ 1[2-1] \\ 1-0 \end{array} \rightarrow \begin{array}{c} 010101 \\ \downarrow \\ \text{VPN} \end{array}$$

117

Coarse Grained:- Has far larger segments

Paging:- VA is divided into fixed length segments (not necessary ~~at~~ it is divided into code, heap, stack).

The PA is divided up similarly

Each segment is called a page

Mapping is from a page in VA to a page frame in PA

Each Page in VA has VPN $\rightarrow$ Virtual Page Number

Advantages:- Easiest abstraction $\rightarrow$ Flexible

All OS does is find some free space and load the process
OS makes use of per process data structure $\rightarrow$ page table
mappable VA $\rightarrow$ PA

Each process has its unique page table.

The VPNs map to different PFNs unless they share the same memory

PTBR $\rightarrow$ Page Table Base Register $\rightarrow$ Stores location of page table.
Saved in context

The table itself is stored in the memory

VPN	PFN	V	P	Present	Dirty	Ref
-----	-----	---	---	---------	-------	-----

V $\rightarrow$ Valid bit (unused pages are invalid $\rightarrow$ access results in a trap)

P $\rightarrow$ Protection bit (~~RW~~ perms RWX perms)

Present $\rightarrow$ Whether page is in memory

Dirty $\rightarrow$ Whether page has been modified since being loaded

Ref $\rightarrow$ Whether page has been ~~most~~ recently used.

- Access PTBR
→ Index into array of PTE as pointed out by PTBR

TLB:- Translation Lookaside Buffer → Cache for Translations

Locality, both Spatial & Temporal, matter while caching

Hardware Handler:-

On CISC machines, the hardware walks through the whole page table

Software Handler:-

Software handles it entirely on RISC

Hardware simply raises a TRAP

During a TLB miss, a context switch occurs and the program returns back to the instr that had a TLB miss. Otherwise, we might result in an inf loop of not executing the process.

Inside TLB:-

TLB is managed by a fully associative cache.

A typical TLB has 32, 64 or 128 entries.

Hardware searches the entire TLB in parallel.

Other bits:-

Valid:- Entry has a valid translation

Protection:- How a page can be accessed (ROONLY, RW, etc)

address space identifier, dirty bit, etc.

↳ ASID (mapped to by pid)

Processes can share a page

But TLB has limited size

Size of pages:- Naïvest soln is to increase size of pages, but internal fragmentation skyrockets

Common sizes = 4KB or 8KB

Hybrid Approach $\rightarrow$ Paging and Segmentation

Instead of a per process page tables, we have per segment page table

But this results in variable sized page tables, causing external fragmentation, and finding memory for them is hard.

Multi Level Page Tables

Create a tree like page

- (i) Chop up the page table into page sized chunks
- (ii) If an entire page has only ~~one~~ invalid entries, don't allocate the page at all.
- (iii) To track, we use a simple data structure, Page Directory

~~Page directory can specify~~

PDBR:- Page Directory Base Register.

This makes it easier to manage memory and the OS can simply grab the next free page when it needs to grow in size

Inverted Page Table

Instead of a per process page table, why not have 1 large page table and use hashing?

Didn't work out is

Demand Paging:-

Swap data from RAM into main memory when RAM is full.

But how?

leverage swap space:- Enables OS to swap out some pages to swap spaces.

Swap space is the dedicated space in the ^{hard disk} memory which can be used by OS to swap in/out pages.

OS should be able to read and write page sized units from/to swap space.

Present-bit is 0 if in swap space, 1 if in RAM.

Block Frame Number is used to find which block in the swap it belongs to.

Valid bit is used to specify if the page ~~exists~~ has been cached

Page Fault:- The act of accessing a page that does not exist in the physical memory.

Hardware raises an exception

OS has to service the page fault

↳ Uses page fault handler.

The "offending" process is moved to a Blocked state

OS can choose to context switch.

if memory is full:- Some pages have to be moved out of physical memory.

Page Replacement is the process of swapping pages in/out to/from memory.

Page Eviction → Removing a page from the memory into swap

A metric = Avg Mem Access Time = AMAT

$$= (\text{Hit\%} \times T_m) + (\text{Miss\%} \times T_o)$$

↗ % of cache misses.

% of cache hits ↙

↓
Cost of
accessing
memory

↘ Cost of accessing disk

Theoretical Optimal:- Belady's policy → replace the page least likely to be used in the future. Theoretical optimal → only for benchmarking.